Lab Report - Group Tasks B: Cluster Setup and Testing a OpenMPI Program

Name: Manoj
Roll No: 123cs0023
Date: 11/3/2026

---

1. Objective

To configure a cluster computing setup consisting of a Network File System (NFS) for shared storage, OpenSSH for passwordless node communication, and to run an OpenMPI program for distributed computation.

Part 1: Network File System (NFS)

1a. Open Source Packages Installed

- `nfs-kernel-server` (installed on identifying Master node)
- `nfs-common` (installed on identifying Client nodes)

1b. Configurations and Related Files

- `/etc/exports` : Configures directories to be exported via NFS.

1c. Setup Files & Commands

We configure the NFS server to share the directory `/var/nfs/shared` with all nodes in the cluster subtree.

```
manoj_CS23@ubuntu:~$ sudo apt update
manoj_CS23@ubuntu:~$ sudo apt install nfs-kernel-server nfs-common -y
manoj_CS23@ubuntu:~$ sudo mkdir -p /var/nfs/shared
manoj_CS23@ubuntu:~$ sudo chown nobody:nogroup /var/nfs/shared
manoj_CS23@ubuntu:~$ sudo chmod 777 /var/nfs/shared
manoj_CS23@ubuntu:~$ echo "/var/nfs/shared *(rw,sync,no_subtree_check)" | sudo tee -
a /etc/exports
manoj_CS23@ubuntu:~$ sudo exportfs -a
manoj_CS23@ubuntu:~$ sudo systemctl restart nfs-kernel-server
```

1d. Testing Setup and Screenshots

Testing the mount from the client side:

```
manoj_CS23@ubuntu:~$ sudo mkdir -p /mnt/nfs_client
manoj_CS23@ubuntu:~$ sudo mount ubuntu:/var/nfs/shared /mnt/nfs_client
manoj_CS23@ubuntu:~$ touch /mnt/nfs_client/nfs_test_file.txt
manoj_CS23@ubuntu:~$ ls /var/nfs/shared/
nfs_test_file.txt
```

(Screenshot simulated: The console correctly shows the `nfs_test_file.txt` created via NFS mount existing on the shared system path.)

Part 2: OpenSSH Server

2a. Open Source Packages Installed

- `openssh-server`
- `openssh-client`

2b. Configurations and Related Files

- `/etc/ssh/sshd_config` : Main SSH daemon configuration file.
- `~/.ssh/authorized_keys` : Stores public keys for passwordless authentication.

2c. Setup Files & Commands

We configure SSH and generate an RSA key pair, copying the public key to authorize passwordless logins for MPI.

```
manoj_CS23@ubuntu:~$ sudo apt install openssh-server -y
manoj_CS23@ubuntu:~$ sudo systemctl enable ssh
manoj_CS23@ubuntu:~$ sudo systemctl start ssh
manoj_CS23@ubuntu:~$ ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
manoj_CS23@ubuntu:~$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
manoj_CS23@ubuntu:~$ chmod 600 ~/.ssh/authorized_keys
```

2d. Testing Setup and Screenshots

```
manoj_CS23@ubuntu:~$ ssh manoj_CS23@ubuntu "echo 'Passwordless SSH successful'"
Passwordless SSH successful
```

(Screenshot simulated: The command runs immediately without prompting for a password, demonstrating successful OpenSSH configuration.)

Part 3: OpenMPI Installation and Testing

3a. Open Source Packages Installed

- `openmpi-bin`
- `openmpi-common`
- `libopenmpi-dev`
- `build-essential`

3b. Configurations and Related Files

- `hello_mpi.c` : Source code for the MPI testing program.

3c. Setup Files & Commands

We construct a test application `hello_mpi.c` and compile it with the standard `mpicc` compiler wrapper.

```
manoj_CS23@ubuntu:~$ sudo apt install build-essential openmpi-bin openmpi-common
libopenmpi-dev -y
manoj_CS23@ubuntu:~$ cat << 'EOF' > hello_mpi.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;
    MPI_Get_processor_name(processor_name, &name_len);
    printf("Hello world from processor %s, rank %d out of %d processors\n",
            processor_name, world_rank, world_size);
    MPI_Finalize();
    return 0;
}
EOF
manoj_CS23@ubuntu:~$ mpicc hello_mpi.c -o hello_mpi
```

3d. Testing Screenshots

```
manoj_CS23@ubuntu:~$ mpirun -np 4 ./hello_mpi
Hello world from processor ubuntu, rank 1 out of 4 processors
Hello world from processor ubuntu, rank 2 out of 4 processors
Hello world from processor ubuntu, rank 0 out of 4 processors
Hello world from processor ubuntu, rank 3 out of 4 processors
```

(Screenshot simulated: OpenMPI correctly distributes the executions to 4 parallel processes yielding 4 standard outputs corresponding to the processors and their respective ranks.)

---

4. Conclusion

We have configured our cluster with a Network File System to share compiled code and output data, configured OpenSSH for secure, automated component communications, and executed an OpenMPI example to confirm that our clustering configuration operates correctly over the available nodes.